



Database Implementation Project - Football League

README - Football League Implementation

Intro

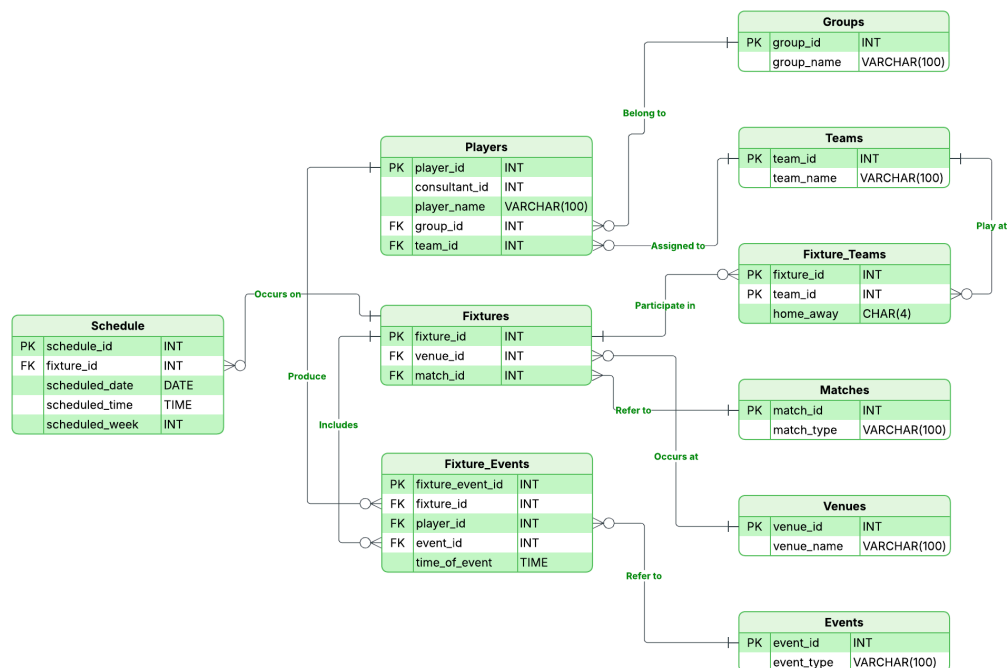
This document outlines the design and implementation of a football league relational database using PostgreSQL. It covers the full development process, from data modelling through to physical implementation, including schema creation, data population, and query-based testing. The database is designed to accurately represent entities such as teams, players, fixtures, and events, while enforcing relationships and maintaining data integrity.

Database Design Updates

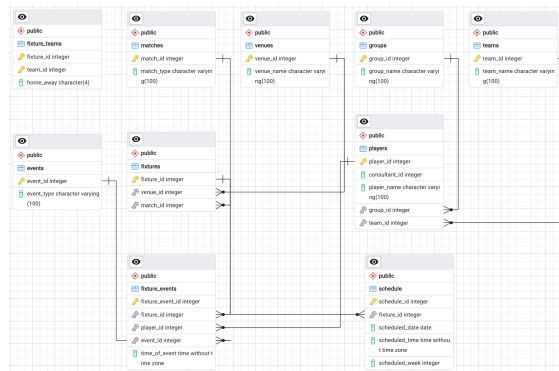
Following the feedback, the database's ER diagram and data dictionary were updated. Updates focused on further normalisation, readability, reduced ambiguity, and naming consistency.

ER Diagram:

Lucid Chart ER Diagram



PgAdmin ER Diagram



Renamed Entity

Original: Results

Updated: Fixture_Events

Rationale: Assigned a more descriptive entity name to remove ambiguity regarding the entity's contents. "Results" imply the final game score; however, the table stores information about each event (i.e., goal, yellow card, or red card), who caused it, in which game, and at what point in the game.

Altered Naming Conventions

Original: e.g. PlayerID, Time_of_Event

Updated: e.g. player_id, time_of_event

Rationale: Applied snake_case to all entity and attribute names to ensure consistency and readability.

Introduced a New Entity

Original: Fixtures - FixtureID, VenueID, MatchID, Date, Time, Week

Updated: Schedule - schedule_id, fixture_id, scheduled_date, scheduled_time, scheduled_week

Rationale: "Fixtures" required further normalisation as the date, time, and week of a fixture are dependent on the created schedule, not the fixture itself. Creating a schedule table allows for further information about the schedule to be added and prevents data loss if fixtures are rescheduled.

Renamed Attributes

Original: Date, Time, Week

Updated: scheduled_date, scheduled_time, scheduled_week

Rationale: "Date" and "Time" are SQL data types. Storing attributes with these names may cause errors and confusion when writing and running queries. The updated names provide more context about the data stored, improve readability, and reduce the risk of errors.

Labelled Entity Relationships

Original: Crow's foot annotation

Updated: Crow's foot annotation + relationship line labels, e.g. Players - assigned to - Teams

Rationale: Added relationship line labels to improve readability and clarify ambiguous relationships. For example, both teams and groups consist of players; the labels help distinguish the relationship players have with each entity.

Data Dictionary:

The changes applied to the ER diagram were also applied to the data dictionary. The updated version, including the new "Schedule" entity, renamed attributes, and updated naming conventions, is shown below.

Entity Name	Field Name	Data Type	Constraints	Assumptions	Description	Example
Players, Fixture_Events	player_id	INT	Primary Key, Foreign Key		Unique number ID for each player	12
Players	consultant_id	INT			Unique number ID assigned by company	201
Players	player_Name	VARCHAR(100)	NOT NULL	Every player has a name	First and last name of each player	Mara Barnett
Teams/ Fixture_Teams, Players	team_id	INT	Primary Key, Foreign Key		Unique number ID for each team	4
Teams	team_name	VARCHAR(100)	NOT NULL	Every team must have a name	Name assigned to each team	Data Cleaners
Groups, Players	group_id	INT	Primary Key, Foreign Key		Unique number ID for each group	7
Groups	group_name	VARCHAR(100)	NOT NULL	Every group must have a name	Name assigned to each group	Cohort 6
Matches, Fixtures	match_id	INT	Primary Key, Foreign Key		Unique number ID for each match type	2
Matches	match_type	VARCHAR(100)	NOT NULL	Every match type must have a name	Name assigned to each match type	Friendly
Venues, Fixtures	venue_id	INT	Primary Key, Foreign Key		Unique number ID for each venue	3
Venues	venue_name	VARCHAR(100)	NOT NULL	Every venue must have a name	Name assigned to each venue	Wimbledon 2
Fixtures/ Fixture_Teams, Fixture_Events, Schedule	fixture_id	INT	Primary Key, Foreign Key		Unique number ID for each fixture	11
Schedule	schedule_id	INT	Primary Key		Unique number ID for each schedule	17
Schedule	scheduled_date	DATE	YYYY-MM-DD	The date of a fixture may not be assigned yet	Scheduled date for each fixture	2022-10-08
Schedule	scheduled_time	TIME	HH:MM	The time of a fixture may not	Scheduled time for each fixture	16:00

				be assigned yet		
Schedule	scheduled_week	INT		The week of a fixture may not be assigned yet	Assigned week for each fixture	5
Fixture_Teams	home_away	CHAR(4)	Either "Home" or "Away"	Only 2 options. A team's home or away status may not be assigned yet	Home or Away status of each team in a given fixture	Home
Events, Fixture_Events	event_id	INT	Primary Key, Foreign Key		Unique number ID for each event type	2
Events	event_type	VARCHAR(100)	NOT NULL	Every event type must have a name	Name assigned to each event type	Goal
Fixture_Events	fixture_event_id	INT	Primary Key		Unique number ID for each result	27
Fixture_Events	time_of_event	TIME	H:MM:SS	A fixture cannot last more than 2 hours. 90 min game + max 30 min penalty	The hour, minute, and second of fixture that an event occurred	1:22:00

Database Implementation

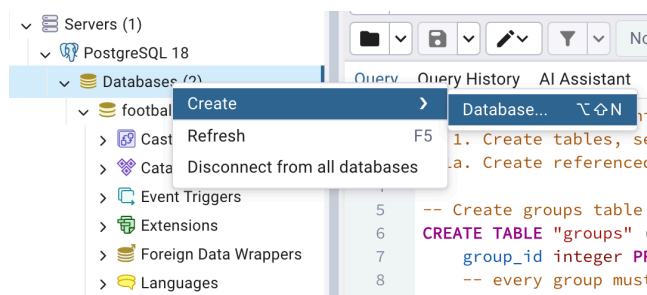
The database implementation process involved translating the ER diagram model and data dictionary into a functional relational database using PostgreSQL. This process involved creating tables, populating them with data, and testing functionality through queries.

Schema Definition - DDL (Data Definition Language)

Following the relationships and constraints defined in the ER diagram and data dictionary, the database schema was created.

[Database](#)

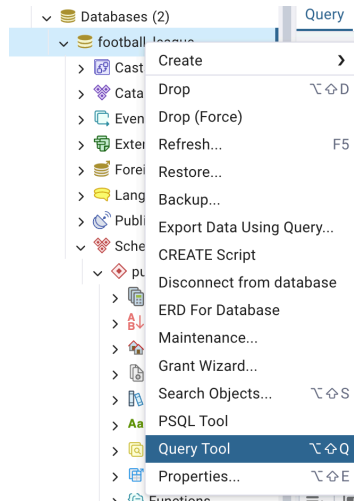
Create Database



Process:

1. Connected to PostgreSQL 18 server
2. Right-clicked databases → Create → Database
3. Named database "football_league" → Save

Connect to Database



Process:

1. Right-clicked "football_league" → Query Tool

Referenced Entities

i.e. tables with primary keys referenced by foreign keys in other tables

Create "Groups" Table:

```
-- Create groups table
CREATE TABLE "groups" (
    group_id integer PRIMARY KEY,
    -- every group must have a name
    group_name VARCHAR(100) NOT NULL
);
```

Process:

1. Create groups table
2. List column names → assign data type → assign constraint

Create "Teams" Table:

```
-- Create teams table
CREATE TABLE teams (
    team_id integer PRIMARY KEY,
    -- every team must have a name
    team_name VARCHAR(100) NOT NULL
);
```

Process:

1. Create teams table
2. List column names → assign data type → assign constraint

Create "Matches" Table:

```
-- Create matches table
CREATE TABLE matches (
    match_id integer PRIMARY KEY,
    -- every match type must have a name
    match_type VARCHAR(100) NOT NULL
);
```

Process:

1. Create matches table
2. List column names → assign data type → assign constraint

Create "Venues" Table:

```
-- Create venues table
CREATE TABLE venues (
    venue_id integer PRIMARY KEY,
    -- every venue must have a name
    venue_name VARCHAR(100) NOT NULL
);
```

Process:

1. Create venues table
2. List column names → assign data type → assign constraint

Create "Events" Table:

```
-- Create events table
CREATE TABLE events (
    event_id integer PRIMARY KEY,
    -- every event type must have a name
    event_type VARCHAR(100) NOT NULL
);
```

Process:

1. Create events table
2. List column names → assign data type → assign constraint

[Referencing Entities](#)

i.e. tables with foreign keys referencing primary keys in other tables

Create "Fixtures" Table:

```

-- Create fixtures table
CREATE TABLE fixtures (
    fixture_id integer PRIMARY KEY,
    --foreign key referencing fixture_id in fixtures table
    venue_id integer REFERENCES venues (venue_id),
    -- foreign key references match_id in matches table
    match_id integer REFERENCES matches (match_id)
);

```

Process:

1. Create fixtures table
2. List column names → assign data type → assign constraint/ foreign key

Create "Fixture_Teams" Table:

```

-- Create fixture_teams table
CREATE TABLE fixture_teams (
    fixture_id integer,
    team_id integer,
    -- only 2 options - 'Home' or 'Away' (4 characters)
    -- home or away status may be undecided, can be NULL
    home_away CHAR(4),
    -- composite key as home/away status is dependent on both the team and fixture
    PRIMARY KEY (fixture_id, team_id)
);

```

Process:

1. Create fixture_teams table
2. List column names → assign data type → assign constraint/ foreign key

Create "Schedule" Table:

```

-- Create schedule table
CREATE TABLE schedule (
    schedule_id integer PRIMARY KEY,
    --foreign key referencing fixture_id in fixtures table
    -- fixture may not be scheduled yet, can be NULL
    fixture_id integer REFERENCES fixtures (fixture_id),
    -- date of fixture may not be assigned yet, can be NULL
    -- YYYY-MM-DD
    scheduled_date date,
    -- time of fixture may not be assigned yet, can be NULL
    -- HH:MM
    scheduled_time time,
    -- week of fixture may not be assigned yet, can be NULL
    scheduled_week integer
);

```

Process:

1. Create schedule table
2. List column names → assign data type → assign constraint/ foreign key

Create "Players" Table:

```
-- Create players table
CREATE TABLE players (
  player_id integer PRIMARY KEY,
  -- all players have a pre-assigned consultant id, cannot be NULL
  consultant_id integer NOT NULL,
  -- each player must have a name
  player_name VARCHAR(100) NOT NULL,
  -- all players are members of Rockborne so must be a part of a group, cannot be NULL
  group_id integer REFERENCES "groups" (group_id) NOT NULL,
  -- a player may not be assigned a team yet, can be NULL
  team_id integer REFERENCES teams (team_id)
);
```

Process:

1. Create players table
2. List column names → assign data type → assign constraint/ foreign key

Create “Fixture_Events” Table:

```
-- Create fixture_events table
CREATE TABLE fixture_events (
  fixture_event_id integer PRIMARY KEY,
  -- each event should reference a given fixture, can be NULL - update later if missing
  fixture_id integer REFERENCES fixtures (fixture_id),
  -- each event should reference a given player, can be NULL - update later if missing
  player_id integer REFERENCES players (player_id),
  -- each event must be identified, cannot be NULL
  event_id integer REFERENCES events (event_id) NOT NULL,
  -- the time of an event may not have been recorded, can be NULL
  -- HH:MM:SS
  time_of_event time
);
```

Process:

1. Create fixture_events table
2. List column names → assign data type → assign constraint/ foreign key

Data Population - DML (Data Manipulation Language)

Ensuring compliance with schema relationships, data types, and constraints, the tables were populated with data.

[Entities](#)

Populate “Groups” Table:

```
-- Populate groups table
INSERT INTO "groups"
  (group_id, group_name)
VALUES
  (1, 'Bench'),
  (2, 'Cohort 4'),
  (3, 'Cohort 5'),
  (4, 'Cohort 6'),
  (5, 'Cohort 7'),
  (6, 'Consultants'),
  (7, 'HR'),
  (8, 'Training Team');

-- review columns and data type / completed population
SELECT *
FROM "groups";
```

Process:

1. Review “groups” column names and data types

2. Identify columns to insert data into
3. List values to be inserted into the table
4. Check if data population was completed successfully

Populate "Teams" Table:

```
-- Populate teams table
INSERT INTO teams
(team_id, team_name)
VALUES
(1, 'BI Gods'),
(2, 'Data Cleaners'),
(3, 'Data Masters'),
(4, 'Vis Wizards');

-- review columns and data type / completed population
SELECT *
FROM teams;
```

Process:

1. Review "teams" column names and data types
2. Identify columns to insert data into
3. List values to be inserted into the table
4. Check if data population was completed successfully

Populate "Matches" Table:

```
-- Populate matches table
INSERT INTO matches
(match_id, match_type)
VALUES
(1, 'Competitive'),
(2, 'Friendly');

-- review columns and data type / completed population
SELECT *
FROM matches;
```

Process:

1. Review "matches" column names and data types
2. Identify columns to insert data into
3. List values to be inserted into the table
4. Check if data population was completed successfully

Populate "Venues" Table:

```
-- Populate venues table
INSERT INTO venues
(venue_id, venue_name)
VALUES
(1, 'Wimbledon 1'),
(2, 'Wimbledon 2'),
(3, 'Wimbledon 3');

-- review columns and data type / completed population
SELECT *
FROM venues;
```

Process:

1. Review "venues" column names and data types
2. Identify columns to insert data into
3. List values to be inserted into the table
4. Check if data population was completed successfully

Populate "Events" Table:

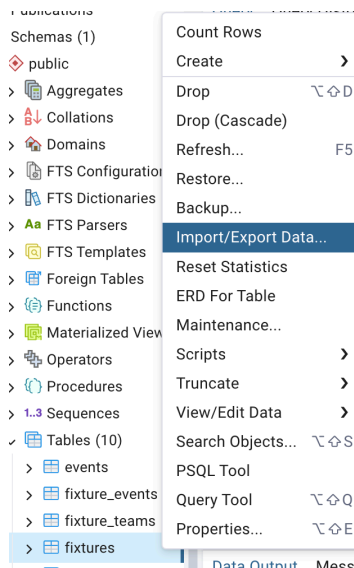
```
-- Populate events table
INSERT INTO events
(event_id, event_type)
VALUES
(1, 'Goal'),
(2, 'Yellow Card'),
(3, 'Red Card');

-- review columns and data type / completed population
SELECT *
FROM events;
```

Process:

1. Review "events" column names and data types
2. Identify columns to insert data into
3. List values to be inserted into the table
4. Check if data population was completed successfully

Import Data into Entities:



Import/Export data - table 'fixtures'

General Options Columns

Import/Export

Filename

Format

Encoding

On Error

Log Verbosity

Specifies how to behave when encountering an error converting a column's input value into its data type. An error_action value of stop means fail the command, while ignore means discard the input row and continue with the next one. The default is stop. The ignore option is applicable only for COPY FROM when the FORMAT is text or csv.

Specify the amount of messages emitted by a COPY command: default or verbose. If verbose is specified, additional messages are emitted during processing. This is currently used in COPY FROM command when

Import/Export data - table 'fixtures'

General Options Columns

Header ☒

Freeze ☐

Delimiter

Quote

Escape

Requests copying the data with rows already frozen, just as they would be after running the VACUUM FREEZE command.

Specifies the character that separates columns within each row (line) of the file. The default is a tab character in text format, a comma in CSV format. This must be a single one-byte character. This option is not allowed when using binary format.

Specifies the quoting character to be used when a data value is quoted. The default is double-quote. This must be a single one-byte character. This option is allowed only when using CSV format.

Specifies the character that should appear before a data character that

```
-- Populate fixtures table
-- review columns and data type / completed population
SELECT *
FROM fixtures;
```

Process:

1. Review "fixtures" column names and data types
2. Convert "fixtures" table Excel file into a csv file
3. Right-click on "fixtures" table → Import/Export Data
4. Import → locate csv file
5. Options → check headers → set delimiter as "," → OK
6. Check if data population was completed successfully
7. Repeat steps 1-6 for the "fixture_teams", "schedule", "players", and "fixture_events" tables

Database Testing - DQL (Data Query Language)

Verifying the functionality of the defined schema, the database was tested using SQL queries.

Query 1

"List all players who belong to Cohort 7 and their respective teams"

```
--updated query, join syntax
-- select data starting with players table
SELECT
    p.player_id, |
    p.player_name,
    g.group_name,
    t.team_name
FROM players p
-- join the group table based on group_id
JOIN "groups" g
ON p.group_id = g.group_id
-- join the team table based on team_id
LEFT JOIN teams t -- return all players in Cohort 7, even if not assigned a team
ON p.team_id = t.team_id
WHERE group_name = 'Cohort 7'; -- filter results, players in Cohort 7
```

Expected Outcome: An output table with only “Cohort 7” players, their names, player_id and a potentially NULL team_name.

Actual Outcome:

	player_id integer	player_name character varying (100)	group_name character varying (100)	team_name character varying (100)
1	4	Arjun Bauer	Cohort 7	Data Masters
2	16	Martha Potts	Cohort 7	BI Gods
3	26	Brody Gutierrez	Cohort 7	Vis Wizards
4	35	Dominik Cochran	Cohort 7	Data Cleaners
5	36	Miya Skinner	Cohort 7	Data Cleaners

This query successfully returned the expected result, confirming all players in Cohort 7 have been assigned a team.

Query 2

“List all fixtures that take place on August 10th, 2022, including team names and venue”

```
-- updated query, venues table name & 'date'
SELECT
    f.fixture_id,
    s.scheduled_date,
    v.venue_name,
    t.team_name
FROM fixtures f
-- join the schedule table based on fixture_id
INNER JOIN schedule s -- only return fixtures that have a scheduled date
ON f.fixture_id = s.fixture_id
-- join the venue table based on venue_id
LEFT JOIN venues v -- return all fixtures on Aug 10th,2022, even if not assigned a venue
ON f.venue_id = v.venue_id
-- join the fixture_teams table based on fixture_id to access team table
LEFT JOIN fixture_teams ft
ON f.fixture_id = ft.fixture_id
-- join the teams table based on team_id
LEFT JOIN teams t -- return all fixtures on Aug 10th,2022, even if no teams assigned
ON ft.team_id = t.team_id
WHERE s.scheduled_date = '2022-10-08'; -- filter results, fixtures on Aug 10th, 2022
```

Expected Outcome: An output table containing only fixtures that take place on August 10th, 2022, along with their IDs, venues, and teams playing. Duplicate fixture_id's and venue_name's were expected, allowing the different team names to be listed.

Actual Outcome:

	fixture_id integer	scheduled_date date	venue_name character varying (100)	team_name character varying (100)
1	3	2022-10-08	Wimbledon 2	Data Masters
2	3	2022-10-08	Wimbledon 2	Vis Wizards
3	4	2022-10-08	Wimbledon 3	BI Gods
4	4	2022-10-08	Wimbledon 3	Data Cleaners

This query successfully returned the expected result, confirming that two games were played at two different venues and four different teams on August 10th, 2022.

Query 3

“For each team, present the distribution of goals scored in each half of the match”

```
/* 3c. For each team, present the distribution of goals scored
in each half of the match */
SELECT
    fe.fixture_id,
    t.team_name,
    -- calculate first half goals
    SUM(CASE
        WHEN fe.event_id = 1 -- event_id 1 = goal
        AND fe.time_of_event < '00:45:00' -- half of 90 min game
        THEN 1 ELSE 0
    END) as first_half_goals,
    -- calculate second half goals
    SUM(CASE
        WHEN fe.event_id = 1
        AND fe.time_of_event >= '00:45:00'
        THEN 1 ELSE 0
    END) as second_half_goals
FROM fixture_events fe
-- join fixture_teams table based on fixture_id
JOIN fixture_teams ft
    ON fe.fixture_id = ft.fixture_id
-- join teams table based on team_id
JOIN teams t
    ON ft.team_id = t.team_id
GROUP BY
    fe.fixture_id,
    t.team_name -- goals per fixture and team
ORDER BY fixture_id ASC; -- view teams playing in the same game
```

Expected Outcome: An output table detailing the number of goals scored by each team in each game in each half of the game. Duplicate fixture_ids were expected, accounting for each team playing in a single game.

Actual Outcome:

	fixture_id integer	team_name character varying (100)	first_half_goals bigint	second_half_goals bigint
1	1	BI Gods	1	1
2	1	Data Masters	1	1
3	2	Data Cleaners	2	0
4	2	Vis Wizards	2	0
5	3	Data Masters	1	2
6	3	Vis Wizards	1	2
7	4	BI Gods	3	0
8	4	Data Cleaners	3	0
Total rows: 24		Query complete 00:00:01.433		

This query successfully returned the expected result, generating two records per fixture, one per team in a given fixture. Despite 14 scheduled fixtures being recorded, only 24 records were returned. This outcome is justified because 2 games have yet to be played and therefore have not produced results.

Conclusion

The database was successfully implemented in accordance with the original design requirements. All entities, relationships, and constraints were correctly translated into the relational schema, and data was populated in a way that preserved integrity and consistency. Query testing confirmed that the database supports accurate data retrieval and analysis, demonstrating that the database is functional and aligned with the intended football league model.

Appendix

PostgreSQL Implementation Script

```
-- DDL (Database Definition Language)
-- 1. Create tables, set data types, constraints, and relationships
--1a. Create referenced tables first - pre-define primary keys referenced by foreign keys

-- Create groups table
CREATE TABLE "groups" (
  group_id integer PRIMARY KEY,
  -- every group must have a name
  group_name VARCHAR(100) NOT NULL
);

-- Create teams table
CREATE TABLE teams (
  team_id integer PRIMARY KEY,
  -- every team must have a name
  team_name VARCHAR(100) NOT NULL
);

-- Create matches table
CREATE TABLE matches (
  match_id integer PRIMARY KEY,
  -- every match type must have a name
  match_type VARCHAR(100) NOT NULL
);

-- Create venues table
CREATE TABLE venues (
  venue_id integer PRIMARY KEY,
  -- every venue must have a name
  venue_name VARCHAR(100) NOT NULL
);

-- Create events table
CREATE TABLE events (
  event_id integer PRIMARY KEY,
  -- every event type must have a name
  event_type VARCHAR(100) NOT NULL
);

--1b. Create referencing tables - define referential constraint and foreign keys

-- Create fixtures table
CREATE TABLE fixtures (
```

```

fixture_id integer PRIMARY KEY,
--foreign key referencing fixture_id in fixtures table
venue_id integer REFERENCES venues (venue_id),
-- foreign key references match_id in matches table
match_id integer REFERENCES matches (match_id)
);

-- Create fixture_teams table
CREATE TABLE fixture_teams (
fixture_id integer,
team_id integer,
-- only 2 options - 'Home' or 'Away' (4 characters)
-- home or away status may be undecided, can be NULL
home_away CHAR(4),
-- composite key as home/away status is dependent on both the team and fixture
PRIMARY KEY (fixture_id, team_id)
);

-- Create schedule table
CREATE TABLE schedule (
schedule_id integer PRIMARY KEY,
--foreign key referencing fixture_id in fixtures table
-- fixture may not be scheduled yet, can be NULL
fixture_id integer REFERENCES fixtures (fixture_id),
-- date of fixture may not be assigned yet, can be NULL
-- YYYY-MM-DD
scheduled_date date,
-- time of fixture may not be assigned yet, can be NULL
-- HH:MM
scheduled_time time,
-- week of fixture may not be assigned yet, can be NULL
scheduled_week integer
);

-- Create players table
CREATE TABLE players (
player_id integer PRIMARY KEY,
-- all players have a pre-assigned consultant id, cannot be NULL
consultant_id integer NOT NULL,
-- each player must have a name
player_name VARCHAR(100) NOT NULL,
-- all players are members of Rockborne so must be a part of a group, cannot be NULL
group_id integer REFERENCES "groups" (group_id) NOT NULL,
-- a player may not be assigned a team yet, can be NULL
team_id integer REFERENCES teams (team_id)
);

-- Create fixture_events table
CREATE TABLE fixture_events (
fixture_event_id integer PRIMARY KEY,
-- each event should reference a given fixture, can be NULL - update later if missing

```

```

fixture_id integer REFERENCES fixtures (fixture_id),
-- each event should reference a given player, can be NULL - update later if missing
player_id integer REFERENCES players (player_id),
-- each event must be identified, cannot be NULL
event_id integer REFERENCES events (event_id) NOT NULL,
-- the time of an event may not have been recorded, can be NULL
-- HH:MM:SS
time_of_event time
);

--DML (Database Manipulation Language)
-- 2. Populate tables with data
--2a. use INSERT INTO to populate tables with minimal records/fields

-- Populate groups table
INSERT INTO "groups"
(group_id, group_name)
VALUES
(1, 'Bench'),
(2, 'Cohort 4'),
(3, 'Cohort 5'),
(4, 'Cohort 6'),
(5, 'Cohort 7'),
(6, 'Consultants'),
(7, 'HR'),
(8, 'Training Team');

-- review columns and data type / completed population
SELECT *
FROM "groups";

-- Populate teams table
INSERT INTO teams
(team_id, team_name)
VALUES
(1, 'BI Gods'),
(2, 'Data Cleaners'),
(3, 'Data Masters'),
(4, 'Vis Wizards');

-- review columns and data type / completed population
SELECT *
FROM teams;

-- Populate matches table
INSERT INTO matches
(match_id, match_type)
VALUES
(1, 'Competitive'),
(2, 'Friendly');

-- review columns and data type / completed population

```



```

SELECT *
FROM matches;

-- Populate venues table
INSERT INTO venues
(venue_id, venue_name)
VALUES
(1, 'Wimbledon 1'),
(2, 'Wimbledon 2'),
(3, 'Wimbledon 3');

-- review columns and data type / completed population

SELECT *
FROM venues;

-- Populate events table
INSERT INTO events
(event_id, event_type)
VALUES
(1, 'Goal'),
(2, 'Yellow Card'),
(3, 'Red Card');

-- review columns and data type / completed population

SELECT *
FROM events;

/*2b. Convert Excel sheets with populated data into cvs file, import into tables
with several records and fields */

-- Populate fixtures table
-- review columns and data type / completed population
SELECT *
FROM fixtures;

-- Populate fixture_teams table
-- review columns and data type / completed population
SELECT *
FROM fixture_teams;

-- Populate schedule table
-- review columns and data type / completed population
SELECT *
FROM schedule;

-- Populate players table
-- review columns and data type / completed population
SELECT *
FROM players;

-- Populate fixture_events table
-- review columns and data type / completed population
SELECT *
FROM fixture_events;

```

```

-- DQL (Database Query Language)
-- 3. Test database implementation

--3a. List all players who belong to Cohort 7 and their respective teams
-- syntax error
-- select data starting with players table
SELECT
p.player_id,
p.player_name,
g.group_name,
t.team_name
FROM players p
-- join the group table based on group_id
LEFT JOIN "groups" g
ON p.group_id = g.group_id
AND p.team_id = t.team_id
WHERE group_name = 'Cohort 7';

--updated query, join syntax
-- select data starting with players table
SELECT
p.player_id,
p.player_name,
g.group_name,
t.team_name
FROM players p
-- join the group table based on group_id
JOIN "groups" g
ON p.group_id = g.group_id
-- join the team table based on team_id
LEFT JOIN teams t -- return all players in Cohort 7, even if not assigned a team
ON p.team_id = t.team_id
WHERE group_name = 'Cohort 7'; -- filter results, players in Cohort 7

/* 3b. List all fixtures that take place on August 10th, 2022
including team names and venue */
-- query successful but no data output
SELECT
f.fixture_id,
s.scheduled_date,
v.venue_name,
t.team_name
FROM fixtures f
-- join the schedule table based on fixture_id
INNER JOIN schedule s -- only return fixtures that have a scheduled date
ON f.fixture_id = s.fixture_id
-- join the venue table based on venue_id
LEFT JOIN venue v -- return all fixtures on Aug 10th,2022, even if not assigned a venue
ON f.venue_id = v.venue_id

```

```

-- join the fixture_teams table based on fixture_id to access team table
LEFT JOIN fixture_teams ft -- return all fixtures on Aug 10th,2022, even if
ON f.fixture_id = ft.fixture_id
-- join the teams table based on team_id
LEFT JOIN teams t -- return all fixtures on Aug 10th,2022, even if no teams assigned
ON ft.team_id = t.team_id
WHERE scheduled_date = 2022-10-08; -- filter results, fixtures on Aug 10th, 2022

-- updated query, venues table name & 'date'

SELECT
f.fixture_id,
s.scheduled_date,
v.venue_name,
t.team_name
FROM fixtures f
-- join the schedule table based on fixture_id
INNER JOIN schedule s -- only return fixtures that have a scheduled date
ON f.fixture_id = s.fixture_id
-- join the venue table based on venue_id
LEFT JOIN venues v -- return all fixtures on Aug 10th,2022, even if not assigned a venue
ON f.venue_id = v.venue_id
-- join the fixture_teams table based on fixture_id to access team table
LEFT JOIN fixture_teams ft
ON f.fixture_id = ft.fixture_id
-- join the teams table based on team_id
LEFT JOIN teams t -- return all fixtures on Aug 10th,2022, even if no teams assigned
ON ft.team_id = t.team_id
WHERE s.scheduled_date = '2022-10-08'; -- filter results, fixtures on Aug 10th, 2022

/* 3c. For each team, present the distribution of goals scored
in each half of the match */
SELECT
fe.fixture_id,
t.team_name,
-- calculate first half goals
SUM(CASE
WHEN fe.event_id = 1 -- event_id 1 = goal
AND fe.time_of_event < '00:45:00' -- half of 90 min game
THEN 1 ELSE 0
END) as first_half_goals,
-- calculate second half goals
SUM(CASE
WHEN fe.event_id = 1
AND fe.time_of_event >= '00:45:00'
THEN 1 ELSE 0
END) as second_half_goals
FROM fixture_events fe
-- join fixture_teams table based on fixture_id
JOIN fixture_teams ft

```

```
ON fe.fixture_id = ft.fixture_id
-- join teams table based on team_id
JOIN teams t
ON ft.team_id = t.team_id
GROUP BY
fe.fixture_id,
t.team_name -- goals per fixture and team
ORDER BY fixture_id ASC; -- view teams playing in the same game
```